Choir

1: Introduction

The conventional encounter with art has for long been marked by an invisible barrier; a distance, both physical and intellectual, most clearly and visibly displayed on the white cube's "Do Not Touch" sign.

The place of the visitor is not to touch or interact with the art; it is to view it from afar, contemplating what might have been the intention behind the painter's brush strokes or the sculptor's chisel dent or the modern elitist artist's conceptual-abstract-process-oriented-ideological-art-you-should-definetely-not-question. In a way, the role of the visitor is to please the whims of the artists and the gallerists and the art intellectuals.

This distance is not natural but has been constructed, and the 20th century has increasingly challenged it. The avant-garde movements of the 1950s and 1960s, such as Fluxus, Situationism and participatory art, had already posed the politically urgent question: who does art belong to? Who has the right to access and produce it? These movements sought to break down the barrier between artist and audience through performance, happenings and collective action, transforming art.

This transformation took shape after World War II, when technology had an ambivalent reputation. On the one hand, it had the optimistic approval of the public, who placed their hopes for reconstruction in it, but on the other hand, it carried with it the shadow of the atomic bomb and the Cold War. Machines were therefore both a promise and a threat, a tool of liberation and control.

Cybernetics was born out of this dual scenario. Norbert Wiener emerged with a revolutionary new vision of machines. They were no longer just passive tools that carried out orders, but systems capable of receiving information from the environment, processing it and modifying their behaviour accordingly. This led to the concept of the feedback loop and with it the possibility of turning a system's response (an output) into input for a new cycle, thus outlining the possibility of creating systems capable of receiving information, giving responses and using these as possible feedback. This led to the realisation that we had a system that, in some way, listened to and received external messages.

The artistic avant-garde and artists who understand the potential of these tools grasp the enormous range of new possible scenarios they can work on. If there is the possibility that a machine can respond, then it is possible to construct a work of art that responds. A work that is not fixed or static but changes according to who is observing it, how they move and what they do. This is how the first great insight of New Media Art was born: not to show, but to engage.

The most radical realisation of the potential of technology applied to art came in the 1970s with Myron Krueger and his installation Videoplace. Krueger built something completely new: an invisible interface. There were no buttons to press or screens to touch, just a space where the silhouettes of each viewer, captured by a camera, were projected onto a screen and related to graphic elements that reacted to their movements in real time.

By adding computer and technological elements of this kind, Krueger is certainly helping to shape the idea that works of art can and should be interactive from now on, and no longer static and detached as they once were, but he is also adding physicality to the work of art itself. He is therefore stating that the viewer's body is already sufficient to inhabit and experience a work without the need for monitors or external devices that translate gestures into commands. The body thus becomes, in a way, the cursor of the artwork, and both communicate with the same language.

In his book The Emancipated Spectator, Rancière criticises the idea that being a spectator is a passive practice, where we're only supposed to take in the artwork and speaks of the importance of the viewer to process that work critically, drawing their own conclusions even if they differ from those of the artist. He speaks against the subjugation of the spectator by the artist, arguing that they are equals.

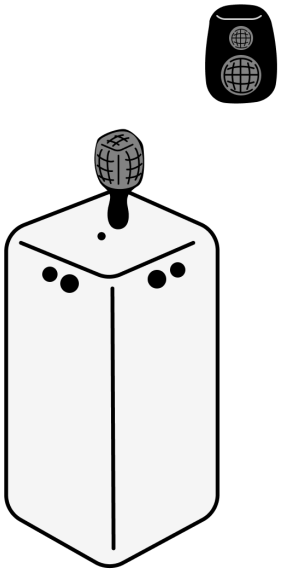
Just like Rancière realised being a spectator does not have to be passive, we argue that they don't have to be separate from the artwork. Just like the artist should not impose a view or interpretation of their work onto the viewer, they should also not forcibly exclude them for the piece.

The interface is no longer a predecessor of the artwork, but an integral part of it. The valorisation of the interface is not so much a result of technological development, but the natural result of the search for artistic innovation and the rise of political and philosophical questions about the place and meaning of the medium and mechanism in art, through which it has risen on top not just as a decoration or structure, but as a part with meaning in itself.

The future of New Media will be defined not by the impressiveness of the technologies that will be invented, but by the space we give them to collide and create with the body, with society and with our politics. The interface is not a means to access art, but the place where art is made.

2: Concept and Methodology

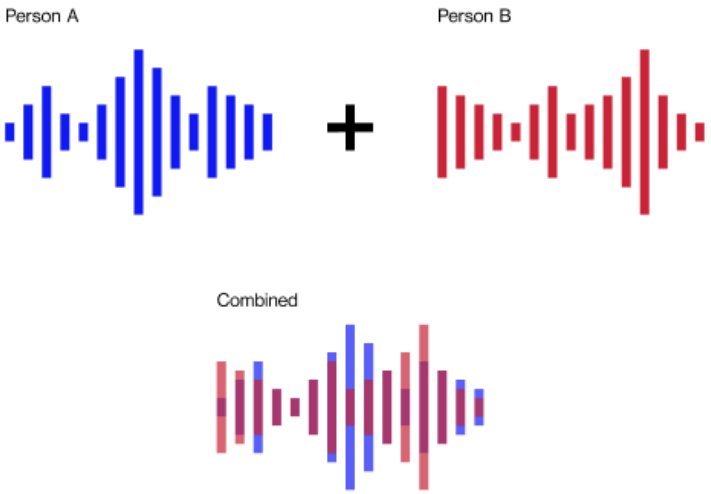
Concept and Theoretical Framework



Initial setup sketch

Our project captures the voices of the audience and replays them in a loop, building layers as more and more people contribute to this “choir” and become a part of the exhibition. Conceptually, the installation is rooted in the principles of Relational Design. The goal is to create a sense of community and belonging, transitioning from a passive and observing audience to an audience that co-creates the exhibition and the installation itself.

We had considered altering the recorded voices; fragmenting them and reorganizing the parts by pitch, to form a choir, or generatively, but chose the mechanical looping system for two main reasons: to ensure the preservation of the individual identity of each user's voice, while still dissolving it into a much larger collective acoustic body, and to avoid the risk of alienating the participant with overly fragmented voices that would only convey the idea of noise. The artwork thus becomes a sort of living archive of temporary presences, making the invisible bonds between visitors physically audible and visually tangible.



Interaction Design Methodology

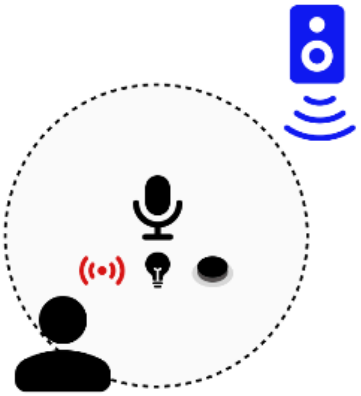
Transforming our concept into a physical experience requires structuring the interaction design around a precise sensory journey that guides the user from the initial observation of the installation to active participation with the project. The design therefore avoids complex instructions, relying instead on intuitive affordances and sensory feedback.

A distance sensor senses the proximity of the spectator and activates a light bulb next to the microphone when they are in close distance, inviting them to further inspect the object and interact with it. When the audience presses the button next to the microphone, it stops the speakers and activates the microphone, which captures the audience's voice and sends it to the Processor. The Processor compiles them and instructs the Arduino to play them in a loop through the Speakers.

The interaction between the user and the installation flows through three distinct states. In the initial "Standby" state,the system actively scans the environment using an ultrasonic distance sensor. The second state, the "Invitation", is triggered when a spectator crosses a physical threshold, approaching within one meter of the artwork. In response, a physical button is illuminated by an LED. This visual cue transforms the interface from an invisible spatial boundary into a tangible point of contact, inviting intentional interaction. The final state, the "Action and Integration" phase, requires the user to press and hold the button to record their voice. The choice to introduce the button into the installation was neither random nor strictly necessary, but was included to invoke intentionality. Indeed, the user is not recorded by accident or against their will; they choose how and when to take part in this system by holding down the button. Upon its release, the system stops recording, the LED flashes to confirm the completion of the action, and the newly captured voice is immediately integrated into the acoustic loop.

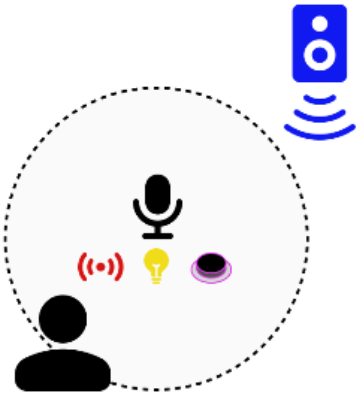
1. setup

2. distance sensor activated

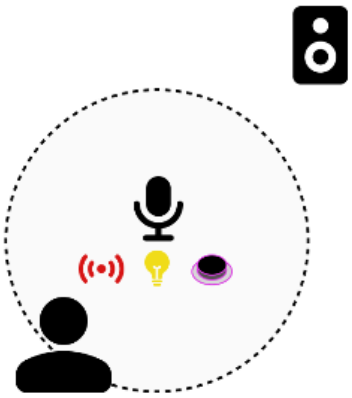


3. light bulb activated

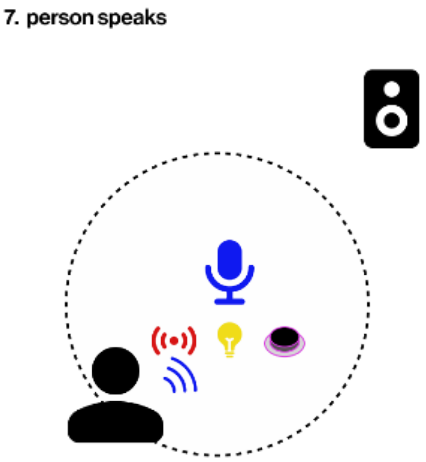
4. button pressed

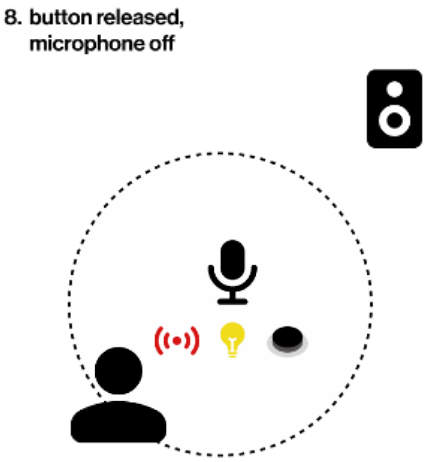


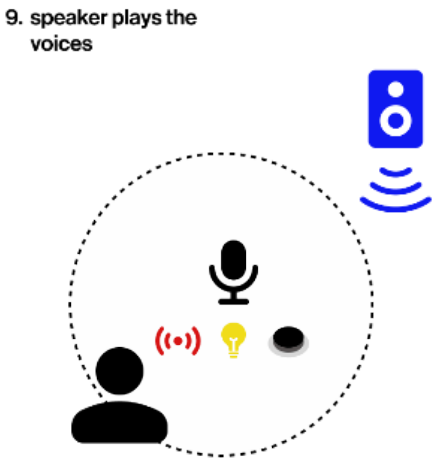
5. speakers off



6. microphone activated







Implementation Strategy and Prototyping

The prototype's architecture strictly divides the tasks: reading the physical inputs is separated from the graphical and audio processing. Dividing the workload in this manner ensures stability and excellent acoustic performance.

Necessary materials:

- 4x Sensor HC-SR04 (distance sensor)
- 1x LED
- 1x Button
- Computer
- Microphone
- Speaker

The physical interface acts as the nervous system of the artwork and is governed by an Arduino Uno microcontroller. The hardware comprises an HC-SR04 ultrasonic sensor to detect spatial proximity, a standard arcade-style push button to register intentionality, and a single LED (paired with a 220-ohm resistor) to provide visual feedback. Arduino's sole responsibility is to process these physical inputs and transmit simple serial commands ("R" to record, "S" to stop) to the main processor.

The cognitive and generative center of the installation is managed by a computer running Processing. Upon receiving the serial commands from Arduino, Processing utilizes the Sound library to access the connected microphone and record the audio input. The software then saves the audio into an array and plays it in a continuous loop. Concurrently, Processing maps the addition of each new audio file onto the visual interface, drawing an expanding network of interconnected particles that visually represent the growing choir.

The light bulb should be activated whenever the sensors capture proximity and microphone should only record if the button is still being pressed.

Initially, and in the first sketches, the speakers were separate from the computer, but due to resource management we decided to use the computer’s speakers.

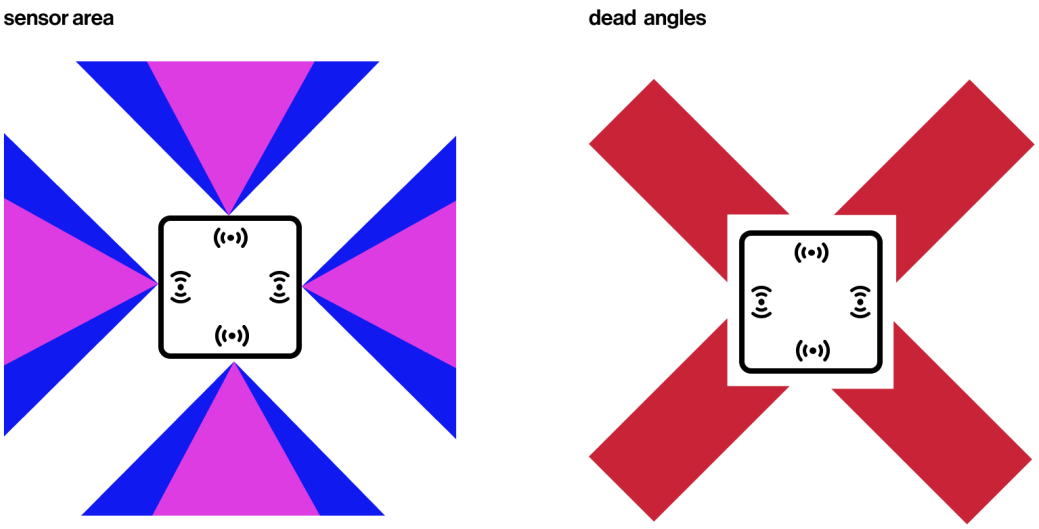
Sensors

Initially we were going to use one singular sensor but we realised that would only capture the movement of a small part of the space around our setup: they capture the movement within a 45º angle, though they work more accurately within 30º; therefore we will need to have four distance sensors, one on each side of the setup. This will result in some dead angles where movement will not be sensed, which is necessary so that the sensors do not interfere with each other. The sound of the speakers should not interfere with the sensors, since the sensors work with ultrasonic sound and the speakers do not emit that. Since the sensors work within a 30 meter distance, we can program the light bulb to light up anywhere within that distance.

Initially, we had not set a “break” in the condition in the code that checks the sensors, which meant that the program was only taking into consideration the information of the last sensor, and ignoring all the 3 previous ones; for instance, if sensor 1 was being triggered but sensor 3 (the last sensor) was not, the program would read all the sensors one by one but only move forward with the information from sensor 3, and say that there was no object being detected. To fix this, we added a “break”, that makes the program exit this loop as soon as it detects that one sensor is being triggered.

```
if (distance > 1 && distance < 100) {
      anObjectIsClose = true;
      d = distance;
      n = i;
      break;
    }
```

After mounting the box in which the setup is stored and presented, we ran into an unexpected difficulty; we realised that if the sensors are not perfectly vertical, the slight downward tilt makes it so that any object – like the table it sits on – constantly triggers the sensors and doesn't allow the LED to turn off. Therefore, we adjusted their position so this error does not persist.



Sound treatment

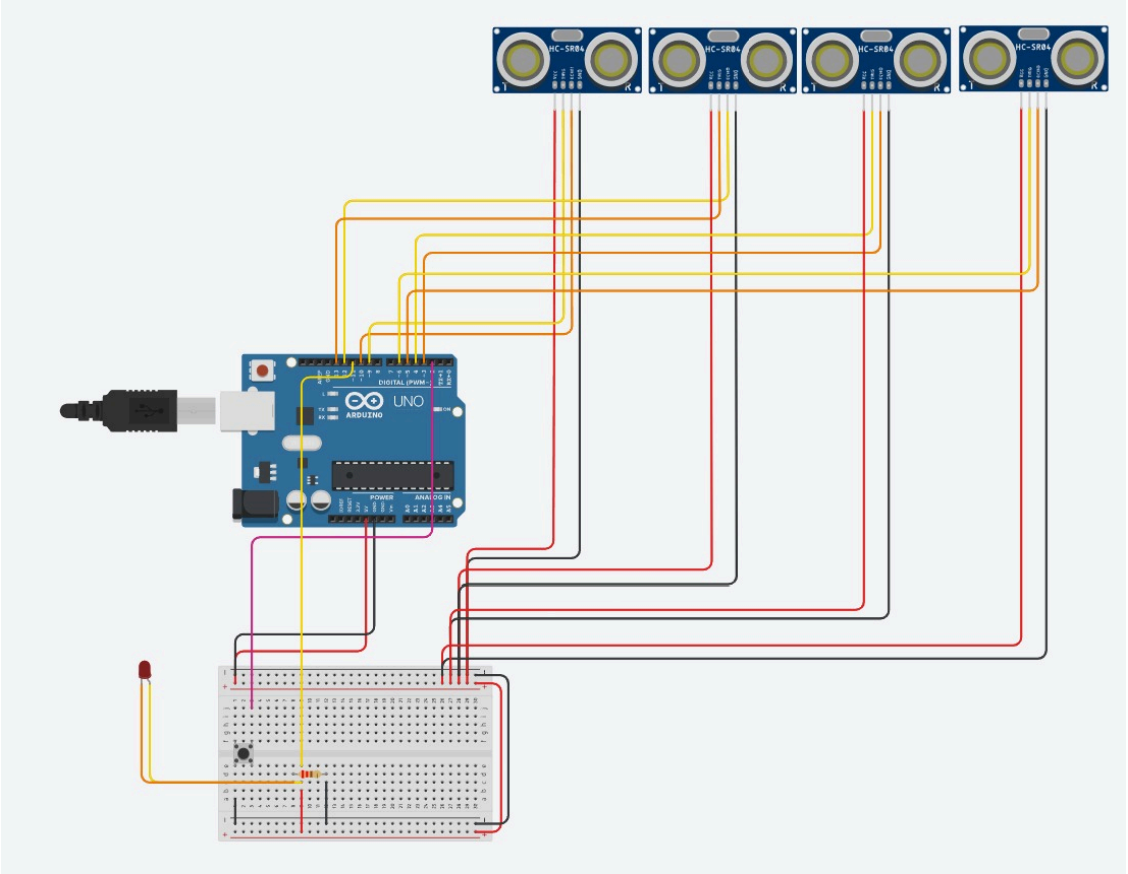
To prevent an excessive overlay of voices, which would sound too chaotic and noisy, it is important to limit the amount of voices played at a time. Each time someone presses the button and speaks to the microphone, their input should be logged into the Processor, and once we reach the pre-established limit of voice inputs we can follow one of two following options:
1. The oldest recorded voice is deleted and the speakers stop playing it.
2. The speakers play the pre-established amount of voices, switching them randomly (except for the most recent voice, that should be played at least one time after being recorded).

To stay coherent and true to our concept, we decided to keep everyone's voice – as not to exclude anyone – and adopted the second option. Later we adapted it and instead of playing the voices completely at random, we established a system of probability in which the more recent voices have a higher likelihood of being played. This means all participants' voices are still being integrated in the project, but the people who recently spoke will hear their voice more often and recognise it as a part of the artwork.

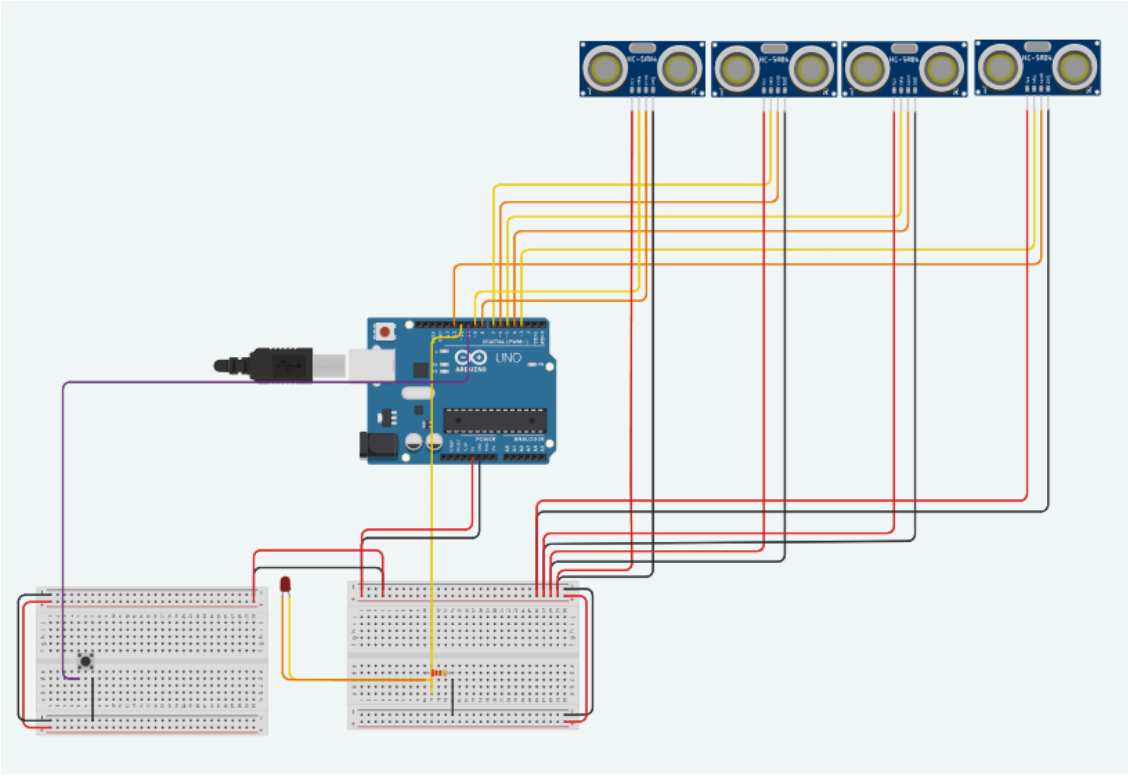
3: Prototype

The system is composed of two separate parts, which are connected to a computer: the Arduino circuit and the microphone. Ideally there should also be external speakers, to create a more immersive experience.

Circuit



Initial circuit, made on Tinkercad



Current circuit, made on Tinkercad

Since the components of our circuit – the LED, the sensors and the button – are going to be presented in a box, with the cables, breadboards and microcontroller hidden away, we extended the LED and the sensors with cables, instead of connecting them directly on the breadboard. Later we also connected the button in its own separate breadboard, to facilitate its integration in the setup.

Microphone

We used a Shure SM-58 microphone, which we connected to our computer. It is important that the microphone is set as the default sound input system in the computer's system settings, so that the Processing – which controls the microphone and does the sound treatment – recognises it as the sound input.

Once the button is pressed, the Arduino sends the message string "Record" to the Processing, which clears the speakers and starts registering the sound the microphone is receiving – because truly, the microphone is always receiving the sound, it just isn't always being recorded.

Arduino Code

It has a boolean which registers whether an object is within the established distance from the sensors. This distance is read inside a for() cycle in the void loop(); it checks every sensor until it detects proximity. When it does, it updates the boolean and exits the cycle. The break function was added because since the cycle was always running, when someone was close to the sensor the boolean was constantly being updated as true by one sensor and right after as false by the other three, which was disrupting the system.

It is also possible to read which sensor is being activated and the distance of the subject to it. This section was commented to reduce the amount of information sent from the Arduino and prevent overload and errors.

When the button is pressed it writes the string "Record". This is later read by the Processing, which executes several actions based on this message. When the button stops being pressed, the Arduino writes yet another message: "Stop Recording". This means the Processing should only record while the button is *being pressed.*

In order for the program to work correctly, the serial monitor in Arduino IDE should be disabled, otherwise the message "Error opening serial port COM8: Port busy" will be displayed, indicating that since the Arduino IDE is already receiving the information from that port, Processing cannot do the same.

Full Arduino code:

```
const int NUM_SENSORS = 4;
int trigPins[NUM_SENSORS] = { 9, 7, 5, 3 };
int echoPins[NUM_SENSORS] = { 8, 6, 4, 12 };
const int buttonPin = 10;
```

```
const int LEDPin = 11;
int lastButtonState = HIGH;
long duration;
float distance;

void setup() {
  Serial.begin(9600);
  pinMode(LEDPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  for (int i = 0; i < NUM_SENSORS; i++) {
    pinMode(trigPins[i], OUTPUT);
    pinMode(echoPins[i], INPUT);
  }
}

void loop() {
  bool anObjectIsClose = false;
  float d = -1;
  float n = -1;

  for (int i = 0; i < NUM_SENSORS; i++) {
    digitalWrite(trigPins[i], LOW);
    delayMicroseconds(2);
    digitalWrite(trigPins[i], HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPins[i], LOW);

    duration = pulseIn(echoPins[i], HIGH, 18000);
    distance = (duration * 0.034) / 2;

    if (distance > 1 && distance < 100) {
      anObjectIsClose = true;
      d = distance;
      n = i;
      break;
    }
  }

//  Serial.println("d=" + (String)d + " n=" + (String)n);


  digitalWrite(LEDPin, anObjectIsClose ? HIGH : LOW);
```

```
  delay(200);

  int buttonState = digitalRead(buttonPin);
  if (buttonState != lastButtonState) {
    if (buttonState == LOW) Serial.println("Record");
    else Serial.println("Stop Recording");
    lastButtonState = buttonState;
  }
}
```

Processing Code

The Processing code establishes the communication with the Arduino and the microphone. It communicates with the Arduino by downloading its library (processing.serial). Besides that, we also use the minim sound library, to manage the sound processing.

We define several variables to set the amount of recordings we can have, and track the record names and the current record we're handling and whether we are or not recording. We also create lists (arrays) to store the sound recordings and the record names.

In the setup, we prepare the ground for the audio library, establish the communication channel with the Arduino and tell Processing how it should process the data it receives from Arduino, so it can read the messages properly.

In the draw we have a void dedicated to processing the information that comes from the Arduino. This is necessary because the arduino sends the information bit by bit (for instance, it sends a sentence letter by letter), which is far slower than the rate at which the void draw runs in Processing. So we create this special void, that waits until the Arduino "stops speaking".

First, we read the full message that the Arduino is sending us. If it is telling us to record, (which means the button was pressed), we first have to check whether we are already recording or not, to prevent mistakes. If we are not recording yet, the Processing silences the speakers. Afterwards, we prepare for the sound recording; we create the name for the current recording and assign the name to the record. Only afterwards we start recording.

Afterwards, we wait for the message from the Arduino telling us to stop recording, which arrives after the person releases the button. When it tells us to stop, we save the record and update the variables that have suffered changes (boolean weAreRecording and currentRecordNumber). Then, we call the void update choir.

The void update choir begins by clearing all the voices, if there are any playing. Then, we check if we have already surpassed the maximum of voices established in the beginning of the code or not. If we haven't, we simply load all our voice records and play them in a loop. If we have surpassed that number, however, we need to first establish a probability for how likely it is for each record to play. This probability is the implementation of the creative

decision explained earlier in “Sound Treatment”, where we’ve decided “the more recent voices have a higher likelihood of being played”. This part was made with some help from AI, to figure out how we could make the probability prioritize newer records while still keeping a random element.

This system works with a points system, in which the record with the most points is the “winner”, and therefore the one that is selected to be played. Processing goes through all the recordings and checks if their number (probability) is bigger than the highest number. When it finds a record with a higher number, it saves that probability as the highest number and that record as the winner record. This decision, however, is only “final” after it goes through *all of the records*. Once a record is declared the final winner, we lower his probability number, so he is not chosen again in the next round. The variable “maiorValor”, which saves the highest number, is reseted at the beginning of each round.

At last, the void update choir plates the winner record in a loop.

Full Processing code:

```
import ddf.minim.*; // importing sound library
import processing.serial.*; // Comunicação com o Arduino

Minim minim;
Serial myPort; // Communication channel with the Arduino
AudioInput in; // O "ouvido" (our microphone) (Processing.org)
AudioRecorder recorder; // Converts the sound into digital data (Processing.org)

int maxVox= 10; // maximum of voices playing simultaneously
ArrayList<String> recordNameStorage = new ArrayList<String>(); // Saves the name of each record
ArrayList<AudioPlayer> soundStorage = new ArrayList<AudioPlayer>(); // Sound storage; we don't delete any records

int currentRecordNumber = 0; // tracks the current record for labelling
boolean weAreRecording = false; // Inicialmente, marcamos esta booleana como falsa pois ainda não estamos a gravar

void setup(){

  minim = new Minim(this);
  in = minim.getLineIn(Minim.MONO, 512); // We choose mono because it is a lighter file

  String portName = Serial.list()[0]; // building the name of the channel through which we are going to communicate with the Arduino
  // Serial.list() takes a look at all devices connected through USB
  // [0] chooses the first device on the list. It is important to make sure the microphone we’ve connected is set as the default audio input in our device (computer), otherwise the sound recording might fail
```

```
  myPort = new Serial(this, portName, 9600); // Establishing the connection with the Arduino ("this"= this Processing file)
  myPort.bufferUntil('\n'); // This is so Processing waits until the end of the Arduino's messages before it starts processing them

}

void draw (){
}

void serialEvent (Serial myPort){
// Checks for Arduino activity and processes it
//Is necessary because the void draw runs fast and the Arduino is slow

// Reading the message
  String message = myPort.readStringUntil('\n'); // We read the message from the Arduino
  if (message != null) {
    // Por vezes a comunicação falha e a mensagem vem vazia, o que faz o Processing crashar.
    // Este if evita que o programa vá abaixo.
    message = trim(message); // Limpa os espaços vazios e informação adicional desnecessária do Arduino

    if (message.equals("Record") && !weAreRecording){ // checking if the Arduino told us to record
      weAreRecording = true;

      // Silencing the choir
      for (AudioPlayer player : soundStorage) { // we go through all our sound files
        if (player != null) {
          player.close(); // and shut them up
        }
      }
      soundStorage.clear();

      String recordName = "voice" + currentRecordNumber + ".wav"; // Creating the record name
      recordNameStorage.add(recordName); // Assigning the name to the current voice recording

      // Turn on the microphone and start recording:
      recorder = minim.createRecorder(in, recordName);
      recorder.beginRecord();
      println("Recording: " + recordName); // A message to help us keep track of what we're recording
    }
```

```
    else if (message.equals("Stop Recording") && weAreRecording){ // Quando paramos a gravação…
      weAreRecording = false; // atualizar a booleana
      recorder.endRecord(); // parar a gravação
      recorder.save(); // guardar o record no nosso array
      println("Record Saved in Sound Storage"); // Confirmação de que o record foi salvo
      currentRecordNumber ++; // Atualizamos a variável currentRecordNumber para a gravação seguinte

      updateChoir();
    }
  }
}

void updateChoir() {

  for (AudioPlayer player : soundStorage) { // we go through all the recordings
    if (player != null) { // checking if there are any recordings left.
      player.close(); // we stop all the recordings
    }
  }
  soundStorage.clear();

  int totalOfVoices = recordNameStorage.size(); // variável para sabermos quantas vozes gravámos. Definimo-la de acordo com o número de gravações no nosso “Storage”

  if (totalOfVoices <= maxVox) { // if don't have any excess voices yet
    for (int i = 0; i < totalOfVoices; i++) {
      AudioPlayer player = minim.loadFile(recordNameStorage.get(i)); // We load all the voice records to a player
      player.loop(); // and we play them in a loop
      soundStorage.add(player);
    }
  } else{
    float[] prob = new float[totalOfVoices]; // probabilidade de um record tocar
    for (int i=0; i<totalOfVoices; i++){
      //prob = recordNumber * random (0,1); // a probabilidade é maior se o record for mais recente
      prob[i] = i * random(0, 1);
    }

    while (soundStorage.size() < maxVox) { // Enquanto os slots não estiverem todos preenchidos
      float maiorValor = -1; // O maior valor (probabilidade) a bater é -1 (como este valor é muito baixo, qualquer record que esteja a entrar vai ter um valor maior e vai ser selecionado)
      int vencedorIndice = -1; // Como nenhum record tem i=-1, isto significa que ainda não há nenhum vencedor
```

```
    // Procura quem teve a maior pontuação na lista
    for (int i = 0; i < totalOfVoices; i++) { // Percorremos todos os records
      if (prob[i] > maiorValor) { // Se o record que estamos a examinar tiver uma probabilidade maior do que o maior valor atual
        maiorValor = prob[i]; // Então o maior valor passa a ser esse. Esta variável é resetada a cada ciclo for, ou seja, no início do preenchimento de cada slot
        vencedorIndice = i; // Guardamos o índice do record vencedor para que o possamos chamar a seguir para tocar
      }
    }

    // Se encontrámos um vencedor, carregamos o som e "anulamos" a sua pontuação
    if (vencedorIndice != -1) { // Filtro de segurança, para termos a certeza que realmente encontrámos o vencedor
      String ficheiroVencedor = recordNameStorage.get(vencedorIndice); // Guardamos o nome – não só o índice, mas o nome completo – do ficheiro vencedor
      AudioPlayer player = minim.loadFile(ficheiroVencedor); // Carregamos o ficheiro vencedor
      player.loop(); // tocamos o som em loop
      soundStorage.add(player);

      prob[vencedorIndice] = -1;  // Mudamos a probabilidade do record vencedor para -1 para que não seja escolhido novamente para a próxima vaga
    }else { // Safety measure: in case there's an error we leave the while
      break;
    }
   }
  }

  println("Choir updated");
}
```

Setup

The physical construction phase of the prototype represented a fundamental stage in the project's development process, insofar as it allowed the concepts previously defined during the ideation phase to be materialised into a tangible and functional solution. Decisions regarding construction materials were guided by practical, economic, and ease-of-manipulation criteria, taking into account the time constraints and resources available within the academic context in which the project was developed.
For the primary structure of the prototype, kappaline was selected as the base material. Kappaline is a composite material consisting of a central layer of extruded polystyrene foam, coated on both sides by high-grammage cardboard layers. This composition endows it with characteristics particularly suited to rapid prototyping, namely its structural lightness, sufficient rigidity to support low-weight electronic components, and versatility in terms of assembly and adhesion.

Panels measuring 30 by 30 centimetres were chosen, from which the necessary faces were cut to construct a box measuring 30 centimetres in length, 30 centimetres in width, and 15 centimetres in height. These dimensions were carefully calculated to comfortably accommodate all electronic components within the structure — namely the Arduino board, the proximity sensors, the microphone module, and the lamp — whilst simultaneously ensuring sufficient space for the routing and organisation of connecting cables without compromising the mechanical integrity of the assembly.

The assembly process of the box involved several sequential stages that demanded geometric rigour and precision in order to ensure the solidity and stability of the final structure. In an initial phase, the six faces of the rectangular parallelepiped were cut from the kappaline panels, ensuring that the dimensions of each piece adhered to the measurements previously established in the prototype sketch. Following the cutting of the faces, the assembly phase commenced, during which the pieces were joined using adhesive specifically formulated for polystyrene and cardboard materials. With the aim of reinforcing the joints and ensuring the stability of the box against potential torsion or pressure resulting from handling during demonstration and testing sessions, all edges and corners were reinforced with adhesive tape. This procedure, although apparently straightforward, proved decisive for the durability of the prototype throughout the various testing and adjustment phases that followed. The adhesive tape, beyond its structural reinforcement function, also contributed to a cleaner and more uniform finish at the junctions between faces, marginally improving the aesthetic quality of the prototype.

One of the most demanding and technically complex stages of the construction phase was the planning and execution of the various openings required for the integration of the electronic components into the physical structure of the box. Each opening was carefully planned taking into account the ideal location of each component in terms of its functional characteristics and the expected interactions with the user. On each of the four lateral faces of the box, two circular openings were made for the installation of the infrared proximity sensors. The symmetric arrangement of sensors in pairs on each face was intended to maximise the angular detection coverage, ensuring that any user approaching the installation from any direction could be reliably detected. Each pair of sensors was positioned on a distinct lateral face of the box, so that the detection fields of adjacent sensors would complement one another, eliminating potential blind spots around the perimeter of the installation.

When a user approaches the installation and enters the detection radius of one or more sensors, the system interprets this event as a trigger for the activation of the luminous response programmed into the Arduino board. The distribution of sensors across the four sides of the box thus ensures that the interaction experience is consistent regardless of the user's position or direction of approach. Additionally, specific openings were made for the integration of the remaining system components. One opening was dimensioned for the passage of the microphone and Arduino board connecting cables to the exterior of the box, enabling the connection of internal components to a computer for sound recording and reprogramming purposes during testing phases. This opening was strategically positioned on one of the lateral faces least visible to the user, so as to minimise the visual impact of the cables on the overall aesthetic of the installation. Another opening was reserved for the manual interaction button, a fundamental component for the voice capture phase of the system. This button, when pressed by the user, signals to the Arduino that it should initiate the recording of the phrase spoken by the user through the microphone. Its location within the structure was defined in accordance with the principles of usability and interaction

design, namely immediate accessibility on the part of the user and the intuitiveness of the pressing action. The button was positioned at an ergonomically appropriate height and on a lateral face of easy access, ensuring that physical interaction with the system could occur naturally and without the need for additional instructions. Openings were also made for the microphone and the lamp, two components whose correct installation and positioning proved decisive for the overall functioning of the system. The microphone was positioned so as to maximise the capture of the user's voice during the recording phase, whilst the lamp was installed in such a way as to provide visible and impactful illumination as a response to the proximity event detected by the sensors.

Following the completion of the construction and assembly phase of the physical structure, a series of functional tests were carried out to validate the behaviour of the system under conditions approximating real-world use. This testing phase proved indispensable for the identification of issues that, whilst not visible during the construction phase, manifested themselves unequivocally during the operation of the system. The most significant problem identified during the testing phase related to the incorrect positioning of the proximity sensors within the openings made in the lateral faces of the box. It was observed that, following the definitive fixation of the sensors in their respective openings, the modules had been oriented inadequately — specifically, the sensors were tilted downwards rather than projecting horizontally outward from the box, as envisaged in the original design. This incorrect orientation had direct and detrimental consequences for the behaviour of the system: the sensors, pointing towards the ground rather than towards the surrounding space of the installation, continuously detected the floor surface as a nearby obstacle, keeping the lamp permanently activated and preventing the correct functioning of the user proximity detection mechanism.

This issue clearly illustrates the challenges inherent to physical prototyping, namely the discrepancy that frequently arises between conceptual design and practical implementation. The fixation of electronic sensors to kappaline structures demands particular attention to alignment precision and fixation rigidity, given that the mechanical properties of this material — specifically its compressibility — may induce angular deviations in components during the fixation process or during subsequent handling of the prototype. Resolving this problem required careful intervention on the structure, consisting of the repositioning and realignment of each sensor to ensure that the optical axis of emission and reception was oriented horizontally and perpendicularly to the face of the box on which each sensor was installed. To guarantee the stability of the new positioning and prevent future angular deviations, the sensors were secured with a greater quantity of adhesive material and, in some cases, small kappaline support pieces were added to provide a rigid backing that maintained the correct orientation. Following this correction, the sensors behaved as intended, detecting the presence of users only when they effectively approached the installation within the defined detection radius, and allowing the lamp to return to its deactivated state in the absence of proximity stimuli.

The constructed box does not constitute merely a physical support for the electronic components, but plays a fundamental role in the interaction logic of the system. The spatial distribution of the sensors across the four sides of the structure serves a specific purpose within the context of interaction design: the physical and angular separation of the sensors allows each one to preferentially cover a different sector of the surrounding space, giving rise to a directional detection system capable of identifying the approach of users from different angles and distances. This approach is consistent with the principles of immersive interactive experience design, in which the installation is intended to react to human

presence autonomously and contextually, without requiring any deliberate action or prior knowledge of the system on the part of the user. The automatic reaction of the lamp to the user's approach creates a discovery experience that encourages spontaneous exploration of the installation, reinforcing its inviting and participatory character.

The decision to position the sensors at the corners of the box — rather than at the central areas of each face — was equally intentional: by placing them at the extremities of the structure, the angular separation between adjacent sensors is maximised, reducing the likelihood that two sensors simultaneously detect the same user and enhancing the system's capacity to discriminate the direction of approach. This characteristic is particularly relevant in contexts of use in which multiple users may interact with the installation simultaneously, such as exhibition or public demonstration environments. In summary, the physical construction phase of the prototype, despite involving significant technical challenges — most notably the sensor-related issue — resulted in a functional, cohesive structure well suited to the objectives of the project. The solutions adopted throughout this process reflect the capacity for adaptation and creative problem-solving that characterises the iterative development of prototypes within the field of interaction technologies.